مكتب

Homework Assignment

Week 6: Advanced String Operations

Total Points: 100 — Due: Next Class

Name: _____ **Date:** _____

Important Instructions

- Work on your own, but you may ask family members for hints only if stuck
- Write your code in Python *(using Thonny or any online IDE)*
- For setting up your code files, kindly see `instructions.pdf`
- For written answers, use the spaces provided on this sheet
- Show your work for full credit!

1 Part 1: Split and Join Mastery (20 points)

Time estimate: 25 minutes

1.1 Exercise 1.1: Understanding Split Operations (10 points)

For each code snippet, predict the output. Remember: split() returns a **list** of strings!

Code	Output (write the list)

`text = "Python is fun"`
`words = text.split()`
`print(words)`	

`data = "apple,banana,orange"`
`fruits = data.split(',')`
`print(fruits)`	

`url = "www.python.org"`
`parts = url.split('.')`
`print(parts[1])`	

`sentence = "a-b-c-d"`
`letters = sentence.split('-')`
`print(len(letters))`	

1.2 Exercise 1.2: Join Practice (10 points)

Complete this code to join lists of strings with different separators:

```
# Exercise: Join strings with different separators
cities = ['Karachi', 'Lahore', 'Islamabad']

# Join with spaces
space_separated = ___________.join(cities)
print(space_separated)  # Should print "Karachi Lahore Islamabad"

# Join with commas and space
comma_separated = ___________.join(cities)
print(comma_separated)  # Should print "Karachi, Lahore, Islamabad"

# Join with arrow separator
arrow_separated = ___________.join(cities)
print(arrow_separated)  # Should print "Karachi -> Lahore -> Islamabad"

# Join with newlines (each city on new line)
newline_separated = ___________.join(cities)
print(newline_separated)

# Challenge: Join only the first 2 cities with " and "
first_two = ___________.join(cities[___:___])
print(first_two)  # Should print "Karachi and Lahore"
```

Save this code as `week06_ex1_2.py`

2 Part 2: Text Parsing Problem Solving (20 points)

Time estimate: 30 minutes

Scenario: Cricket Score Parser

You're building a program to parse cricket match data. The data comes as comma-separated strings that need to be processed and reformatted.

2.1 Exercise 2.1: Parse Player Data (12 points)

Complete this program that parses and displays cricket player information:

```
# player_parser.py
# Parse and format cricket player data

# Player data format: "Name,Team,Runs,Wickets"
player_data = "Babar Azam,Pakistan,3000,0"

# Step 1: Split the data into parts
parts = player_data.split(_________)
print("Raw parts:", parts)

# Step 2: Extract individual pieces
player_name = parts[___]
team_name = parts[___]
runs = int(parts[___])  # Convert to number
wickets = int(parts[___])

```

```python
# Step 3: Create formatted output
# Format: "Player: NAME | Team: TEAM | Stats: R runs, W wickets"
output = "Player: {} | Team: {} | Stats: {} runs, {} wickets"
formatted = output.format(__________, __________, __________, __________)
print(formatted)

# Step 4: Create a different format with join
# Format: "NAME (TEAM) - R/W"
parts_to_join = [player_name, " (", team_name, ") - ",
                 str(runs), "/", str(wickets)]
compact = __________.join(parts_to_join)
print("Compact:", compact)
```

Save this code as `week06_ex2_1.py`

2.2 Exercise 2.2: Clean Messy Input (8 points)

Users often enter data with extra spaces. Complete this cleaning program:

```python
# data_cleaner.py
# Clean up messy user input

messy_input = "   Too    many    spaces    here   "

# Step 1: Use split() without arguments to remove all extra spaces
words = messy_input.split()  # This removes ALL whitespace!
print("Words found:", words)

# Step 2: Join with single spaces
clean_text = __________.join(words)
print("Cleaned:", clean_text)  # Should print "Too many spaces here"

# Step 3: Try with a sentence
sentence = "Python    is     really      fun"
clean_sentence = __________.join(sentence._________)
print(clean_sentence)  # Should print "Python is really fun"

# Challenge: Clean and make uppercase
messy = "  hello   world  "
result = __________.join(messy.split())._________()
print(result)  # Should print "HELLO WORLD"
```

Save this code as `week06_ex2_2.py`

3 Part 3: Pattern Searching (15 points)

Time estimate: 20 minutes

3.1 Exercise 3.1: Input/Output Matching (8 points)

Match each code snippet with its correct output:

Code	Output

`email = "ali@gmail.com"`
`if "@" in email:`
`  print("Valid")`
`else:`
`  print("Invalid")`
	"Invalid"

`text = "Python is fun"`
`pos = text.find("is")`
`print(pos)`
	"Valid"

`msg = "hello hello"`
`count = msg.count("ll")`
`print(count)`
	7

`word = "programming"`
`pos = word.find("z")`
`print(pos)`
	-1

	2

3.2 Exercise 3.2: Complete the Search Program (7 points)

Fill in the blanks to complete this email validator:

```
# email_checker.py
# Validate email addresses

email = input("Enter your email: ")

# Check 1: Must contain @ symbol
if _______ not in _______:
    print("Error: Email must contain @")
elif email.count("@") > 1:
    print("Error: Too many @ symbols")
else:
    # Check 2: Find the @ position
    at_position = email._______("@")

    # Check 3: @ should not be at start or end
    if at_position == ___ or at_position == len(email) - 1:
        print("Error: @ cannot be at start or end")
    else:
        # Check 4: Should have a dot after @
        domain_part = email[at_position + 1:]
        if "." _______ domain_part:
            print("Valid email format!")
        else:
            print("Error: Domain needs a dot (.com, .pk, etc.)")
```

Save this code as `week06_ex3_2.py`

4 Part 4: String Formatting (20 points)

Time estimate: 25 minutes

4.1 Exercise 4.1: Create a Report (12 points)

Complete this program that creates a formatted student report:

```python
# report_generator.py
# Generate formatted student reports

# Student data
students = [
    "Ali,Math,95",
    "Sara,Math,88",
    "Ahmed,Math,92"
]

print("STUDENT GRADE REPORT")
print("=" * 50)

# Print header with formatting
print("{:<15} | {:^10} | {:>8}".format("Student Name", "Subject", "Grade"))
print("-" * 50)

# Process each student
for student_data in students:
    # Split the data
    parts = student_data.split(_________)
    name = parts[___]
    subject = parts[___]
    grade = parts[___]

    # Print formatted row
    # Name: left-aligned, 15 chars
    # Subject: center-aligned, 10 chars
    # Grade: right-aligned, 8 chars
    print("{:<___} | {:^___} | {:>___}".format(name, subject, grade))

print("=" * 50)
```

Expected output:

```
STUDENT GRADE REPORT
==================================================
Student Name    |   Subject  |    Grade
--------------------------------------------------
Ali             |    Math    |       95
Sara            |    Math    |       88
Ahmed           |    Math    |       92
==================================================
```

Save this code as `week06_ex4_1.py`

4.2 Exercise 4.2: Format Numbers (8 points)

Complete this code to format numbers professionally:

```python
# number_formatter.py
# Practice formatting numbers

# Price formatting (2 decimal places)
price = 49.5
formatted_price = "Price: Rs. {:._____f}".format(price)
print(formatted_price)  # Should print "Price: Rs. 49.50"

```

```python
# Large numbers with commas
population = 220892340
formatted_pop = "Population: {:,}".format(population)
print(formatted_pop)  # Should print "Population: 220,892,340"

# Percentage (1 decimal place)
accuracy = 0.9567
formatted_acc = "Accuracy: {:.1%}".format(accuracy)
print(formatted_acc)  # Should print "Accuracy: 95.7%"

# Aligned numbers in columns
print("{:>10} | {:>10}".format("Item", "Price"))
print("{:>10} | Rs. {:>7.2f}".format("Notebook", 50.0))
print("{:>10} | Rs. {:>7.2f}".format("Pen", 15.5))
print("{:>10} | Rs. {:>7.2f}".format("Eraser", 5.25))
```

Save this code as `week06_ex4_2.py`

5 Part 5: Debug Detective (15 points)

Time estimate: 20 minutes

Each code snippet has errors. Find and fix ALL errors:

5.1 Snippet 1: Split/Join Confusion (5 points)

```python
# This code has 3 errors - fix them all!
sentence = "Python is amazing"
words = sentence.split()
first_word = words.upper()  # Error!
result = "-".split(words)   # Error!
print(result)  # Should print "PYTHON-IS-AMAZING"
```

Fixed version:

```python
# Write the corrected code here:
```

5.2 Snippet 2: Find Method Misuse (5 points)

```python
# This code has 2 errors
text = "Welcome to Python programming"
position = text.find("Java")
# Want to print the word starting at position
print(text[position:position+4])  # This will crash!
```

Fixed version:

```python
# Write the corrected code here:
```

5.3 Snippet 3: Format Mistakes (5 points)

```python
# This has 3 formatting errors
name = "Ali"
age = 13
```

```python
score = 95.678

# Wrong number of arguments
message = "Name: {} Age: {} Score: {}".format(name, age)

# Wrong format specifier
print("Score: {:.1f}%".format(score))  # Should show as percentage

# Alignment confusion
print("{:10>}".format(name))  # Wrong alignment syntax
```

Fixed version:

```python
# Write the corrected code here:
```

Save all corrected versions in ***week06_debug_fixed.py***

6 Part 6: Full Program Development (20 points)

Time estimate: 35 minutes

6.1 Exercise 6.1: CSV Data Processor (20 points)

Create a complete program that processes comma-separated data:

Program Requirements

Your program should:

- Accept multiple lines of CSV data (Name,City,Phone)
- Parse each line using split()
- Validate that each line has exactly 3 parts
- Format and display the data in a neat table
- Count and display statistics

Example Run:

```
=== Contact Management System ===
Enter contact data (Name,City,Phone) or 'done' to finish:
> Ali,Karachi,0300-1234567
Contact added!
> Sara,Lahore,0321-9876543
Contact added!
> Ahmed,Islamabad
Error: Invalid format! Need Name,City,Phone
> Fatima,Karachi,0333-5555555
Contact added!
> done

=== CONTACT LIST ===
Name              | City           | Phone
-------------------------------------------------
```

```
Ali                 | Karachi         | 0300-1234567
Sara                | Lahore          | 0321-9876543
Fatima              | Karachi         | 0333-5555555
================================================
Total contacts: 3
Cities represented: Karachi, Lahore
```

Starter Code:

```python
# contact_manager.py
# Name: ______________
# Date: ______________

# CSV Contact Data Processor

def main():
    contacts = []   # List to store valid contacts
    cities = []     # List to track unique cities

    print("=== Contact Management System ===")
    print("Enter contact data (Name,City,Phone) or 'done' to finish:")

    while True:
        user_input = input("> ")

        # Check if user wants to stop
        if user_input.lower() == "done":
            break

        # Parse the input
        parts = user_input.split(_________)

        # Validate: should have exactly 3 parts
        if len(parts) != ___:
            print("Error: Invalid format! Need Name,City,Phone")
            continue

        # Extract data
        name = parts[___].strip()  # .strip() removes extra spaces
        city = parts[___].strip()
        phone = parts[___].strip()

        # Store the contact
        contacts.append([name, city, phone])

        # Track unique cities
        if city not in cities:
            cities.___________(city)

        print("Contact added!")

    # Display all contacts in formatted table
    print("\n=== CONTACT LIST ===")
    print("{:<18}| {:<15}| {:<15}".format("Name", "City", "Phone"))
    print("-" * 50)

    for contact in contacts:
        name, city, phone = contact
        print("{:<18}| {:<15}| {:<15}".format(name, city, phone))

```

```
    print("=" * 50)

    # Display statistics
    print("Total contacts:", len(contacts))
    print("Cities represented:", ___________.join(cities))

# Run the program
if __name__ == "__main__":
    main()
```

Testing Checklist:

- ☐ Program runs without errors
- ☐ Correctly splits CSV data
- ☐ Validates input (rejects lines without 3 parts)
- ☐ Displays formatted table with proper alignment
- ☐ Tracks unique cities correctly
- ☐ Shows accurate statistics at the end
- ☐ Handles 'done' command to exit

Save this program as `contact_manager.py`

7 Part 7: Reflection Questions (5 points)

Time estimate: 10 minutes

Answer these questions in complete sentences:

1. **When would you use split() in a real program? Give an example from your own life.**

2. **What's the difference between using "in" and find() to search for text? When would you choose one over the other?**

3. **Why is string formatting important for making programs user-friendly? How does it improve your programs?**

8 Debugging Discoveries (5 bonus points)

Growth Mindset: Errors are learning opportunities!

Did you encounter any interesting errors while completing this homework? Share one error and how you fixed it:

My Debugging Story

The Error I Got: _____

What I Tried First: _____

How I Fixed It: _____

What I Learned: _____

No errors? Share a tricky part you figured out instead!

9 Bonus Section: Extra Challenges (Optional - 5 extra points)

Only attempt if you've finished everything else!

9.1 Challenge 1: Advanced Text Parser (3 points)

Create a program that parses and analyzes a paragraph:

```
# text_analyzer.py
paragraph = """Python is amazing. Python is powerful.
Programming with Python is fun."""

# Your tasks:
# 1. Split into sentences (split on ". ")
# 2. Count how many times "Python" appears (use count)
# 3. Find the position of "Programming" (use find)
# 4. Replace "Python" with "Coding" and join back together
# 5. Format output showing:
#    - Number of sentences
#    - Number of times "Python" appears
#    - Modified paragraph
```

9.2 Challenge 2: Smart Join (2 points)

Write a function that joins a list with "and" before the last item:

```
def smart_join(items):
    """Join items with commas and 'and' before last item"""
    # Example: ['a', 'b', 'c'] -> "a, b and c"
    # Hint: Join all but last with ', ', then add ' and ' + last


```

```
7
8
9
10 # Test your function
11 print(smart_join(['Ali', 'Sara', 'Ahmed']))
12 # Should print "Ali, Sara and Ahmed"
```

Submission Checklist

Before submitting, make sure you have:

- ☐ Completed all required sections (Parts 1-7)
- ☐ Saved each code exercise in its own file:
  - `week06_ex1_2.py`
  - `week06_ex2_1.py`, `week06_ex2_2.py`
  - `week06_ex3_2.py`
  - `week06_ex4_1.py`, `week06_ex4_2.py`
  - `week06_debug_fixed.py`
  - `contact_manager.py`
- ☐ Created a folder named `week6_hw_yourname` with all files
- ☐ Tested each code file individually to ensure it runs without errors
- ☐ Verified split() and join() are used correctly
- ☐ Checked that formatting produces neat, aligned output
- ☐ Tested your contact manager with valid and invalid input
- ☐ Written your name at the top of this paper
- ☐ Answered all reflection questions
- ☐ Attempted bonus section (optional)

Module 2 Concepts Check:
Which concept was most challenging? _____
Which string operation (split/join/find/format) do you want more practice with? _____

Time Tracking:
How long did this homework take you? _____ hours

Parent/Guardian Signature: _____
(Confirming student completed work independently)

Great job mastering advanced string operations!

Next week: Lists - your first mutable data structure!